STALENESS-AWARE PREDICTIVE LOAD BALANCER

SA-PLB | Simulation-based research report

A hybrid load-balancing approach that uses machine-learning predictions when monitoring observations are fresh and falls back to Weighted Round Robin (WRR) when they are stale.

Data statement. The dataset is a **hybrid**: the only real, public input is the *shape* of the request load over time, taken from a Zenodo trace. Replicas, queues, monitoring delays, interference and response times are all **simulated**. No value in this report is a measurement from real containers.

Prepared by: [your name] | Institution: [institution] | Supervisor: [supervisor] | Date: [date]

Abstract

Cloud applications face changing workloads, so requests must be spread across replicas. Round Robin (RR) and Weighted Round Robin (WRR) use fixed rules; machine-learning (ML) routers predict response time from monitoring data, but that data can be stale when the routing decision is made. SA-PLB checks the age of every candidate observation and uses ML only if all are within a freshness threshold; otherwise it falls back to WRR. A simulation compares RR, WRR, ML and SA-PLB over three monitoring delays and three interference levels on identical request traces. Staleness hurts plain ML (mean response rises 88% from 0.1 s to 4.0 s delay with no interference). SA-PLB is essentially the same as ML when data is fresh (within 8 ms), is faster than ML in 5 of the 6 stale-data conditions, but is much slower than ML in one (heavy interference with 4 s delay), because the WRR fallback is poor there. WRR is best when there is no interference. The findings are specific to the simulated setup and do not establish general superiority.

1. Introduction and Problem Statement

Load balancing distributes requests across service replicas; the choice of replica affects queueing and response time. RR cycles through replicas, and WRR uses fixed weights for differing capacity. ML-based routing predicts the response time of each candidate and picks the lowest, but the predictions rely on monitoring observations that may no longer describe current replica conditions. **Problem:** an ML router can act on outdated observations during sudden workload changes. **Objectives:** (1) implement RR and WRR; (2) train an ML response-time predictor; (3) compute observation age; (4) use ML only when observations are fresh; (5) fall back to WRR when they are stale; (6) evaluate mean and tail latency, throughput and fallback rate.

2. Proposed Method

2.1 Replicas and policies

Replica	CPU cores	Relative speed	Memory
R0	2	1.0	1024 MB
R1	2	0.9	1024 MB
R2	1	1.1	512 MB

RR cycles through replicas. **WRR** uses smooth weighted round robin with static weights (4, 4, 2) derived from cores x speed, i.e. it knows true static capacity. **ML** predicts response time on every replica and picks the lowest prediction, ignoring freshness. **SA-PLB** uses the same ML only when every candidate observation is fresh; otherwise it uses WRR.

2.2 Freshness check

Observation age = decision time minus the observation's collection time. The decision-level age is the **maximum** age over the candidate observations (one stale candidate makes the comparison unreliable). If it is at most the threshold, SA-PLB uses ML; otherwise WRR. The threshold used in the final evaluation is **2 s**, chosen on validation seeds only.

2.3 ML model

A HistGradientBoostingRegressor predicts log response time (converted back to ms). Features, all available at decision time: observed active requests, queue length, CPU and memory utilisation, recent average response time, request size, request-type cost, replica cores and speed, and expected service time. Observation age is deliberately **not** a feature, so the plain ML baseline does not account for freshness. The true replica state and the request's own response time are never used.

3. Dataset: what was used and how it was produced

This is a hybrid dataset. A public Zenodo trace supplies only the workload *shape*; everything else is generated by the SA-PLB simulator.

Component	Origin	Details
Load shape over time	REAL (public Zenodo file)	Number of records per 1-minute bucket (column index 1), square-root compressed, most bursty 20-minute window. The file's columns are undocumented, so no claim is made about what a record means.
Request arrivals, sizes, types	SIMULATED	Poisson arrivals following the load shape; log-normal sizes; GET/POST/PUT mix.
Replicas and response times	SIMULATED	Exact first-come-first-served multi-server queue per replica: response = queue wait + service time.
Monitoring observations	SIMULATED	Each replica is sampled about every 0.5 s; the sample is the true state at collection time and reaches the balancer after a random delay (plus occasional 5 s stalls).
Interference	SIMULATED	Hidden slowdown episodes (2 to 3.5 times slower for 20 to 60 s) per replica; rates 0, 0.10 and 0.25 per minute per replica.

File	Rows	Content
training_log.csv	47,051	Random-routing logs, 8 periods; chronological split train / val / test by period
requests.csv	6,854	One evaluation scenario (interference 0.10, delay 1.5 s, seed 0)
replica_metrics.csv	30,276	Monitoring observations with collection and delivery times
routing_results.csv	27,416	Per-request decisions for RR, WRR, ML, SA-PLB (4 x 6,854)
field_provenance.csv	12	Origin label (simulated / derived / policy / model output) of every field

Why not a public dataset only? Public traces lack container-level metrics, routing decisions and metric-collection timestamps together, and staleness needs all three. An earlier, separate file (hybrid_dataset.csv) is not used anywhere in this report.

4. Experimental Setup

Setting	Value
Replicas	3
Training / evaluation horizon	20 min per period / 20 min
Peak utilisation (evaluation)	0.85 of capacity
Mean extra monitoring delay	0.1 s, 1.5 s, 4.0 s (mean observation age about 0.6, 1.7, 2.7 s)
Interference (per minute per replica)	0, 0.10, 0.25
Thresholds tried on validation	0.5, 1.0, 2.0, 4.0 s
Validation seeds	100, 101
Test seeds	0, 1, 2 (disjoint from validation and training seeds)
Training	8 random-routing periods under healthy monitoring (0.1 s delay); last period = test, one before = validation

Every policy replays the **same** requests and the **same** monitoring schedule for a given seed, so differences between policies are paired.

5. Results

5.1 Threshold selection (validation seeds only)

The 2.0 s threshold had the lowest validation mean response time. **2.0 s and 4.0 s are almost tied** (627 vs 632 ms), so the choice between them is not strong evidence; 0.5 s and 1.0 s are clearly worse because they fall back to WRR too often.

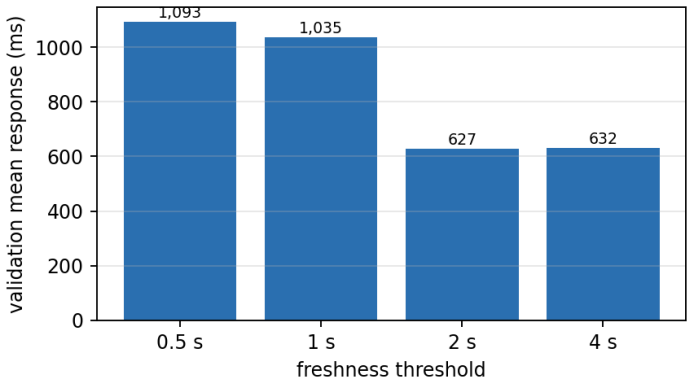


Figure 1. Validation mean response time by freshness threshold (lower is better).

5.2 Policy comparison: mean response time

Mean over the three test seeds, in milliseconds.

Interf.	Delay (s)	RR	WRR	ML	SA-PLB	SA-PLB fallback
0	0.1	1,077	302	397	396	0%
0	1.5	1,077	302	597	495	27%
0	4	1,077	302	748	359	80%
0.1	0.1	1,306	1,014	454	458	0%
0.1	1.5	1,306	1,014	669	586	27%
0.1	4	1,306	1,014	899	553	80%
0.25	0.1	5,940	4,830	627	635	0%
0.25	1.5	5,940	4,830	1,004	929	27%
0.25	4	5,940	4,830	1,595	2,731	80%

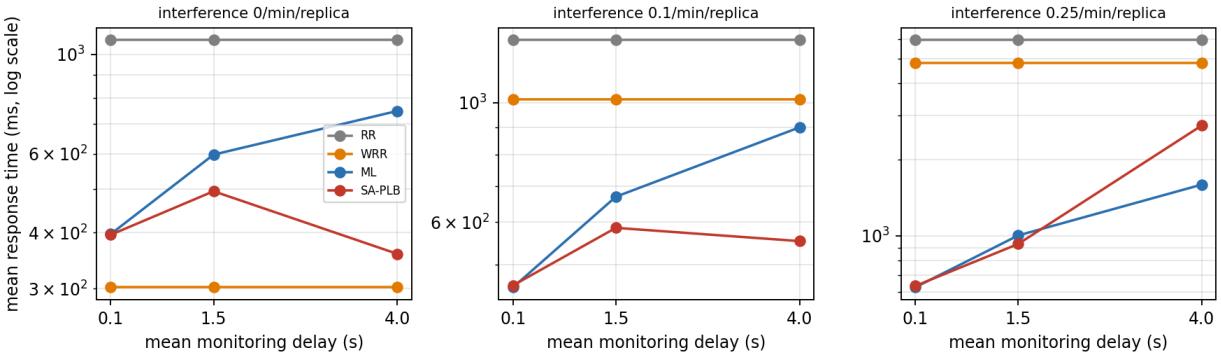


Figure 2. Mean response time by policy and monitoring delay (log scale; RR and WRR do not use monitoring, so they are flat).

5.3 Tail latency and throughput

Delays of 1.5 s and 4.0 s (at 0.1 s SA-PLB and ML are within 8 ms of each other). Milliseconds, mean over test seeds.

Interf.	Delay (s)	Policy	Mean	P95	P99	Throughput (req/s)
0	1.5	WRR	302	690	1,161	5.70
0	1.5	ML	597	2,099	3,907	5.70
0	1.5	SA-PLB	495	1,622	3,074	5.70
0	4	WRR	302	690	1,161	5.70
0	4	ML	748	2,776	5,259	5.69
0	4	SA-PLB	359	938	1,929	5.70
0.1	1.5	WRR	1,014	5,510	10,958	5.70
0.1	1.5	ML	669	2,380	4,340	5.70
0.1	1.5	SA-PLB	586	2,052	3,832	5.70
0.1	4	WRR	1,014	5,510	10,958	5.70
0.1	4	ML	899	3,694	6,303	5.69
0.1	4	SA-PLB	553	1,820	5,564	5.70
0.25	1.5	WRR	4,830	32,439	62,950	5.52
0.25	1.5	ML	1,004	3,886	9,366	5.69
0.25	1.5	SA-PLB	929	3,673	9,378	5.69
0.25	4	WRR	4,830	32,439	62,950	5.52
0.25	4	ML	1,595	7,145	18,879	5.67
0.25	4	SA-PLB	2,731	18,543	45,974	5.57

Throughput is about 5.7 requests/s for every policy except under the heaviest interference, where queues build up (RR 5.45, WRR 5.52, SA-PLB with 4 s delay 5.57). Against ML, SA-PLB's P95 is lower wherever its mean is lower, and so is P99 except one negligible case (0.25 interference, 1.5 s: 9,378 vs 9,366 ms). Where it loses (0.25 interference, 4 s) it loses by more in the tail than in the mean (P95 18,543 vs 7,145 ms; P99 45,974 vs 18,879 ms).

5.4 Paired comparison across seeds

Negative = SA-PLB faster. 'Better in' counts the seeds (out of 3) where SA-PLB had the lower mean. At 0.1 s delay the differences from ML (about 8 ms or less) are noise: even one fallback decision in thousands changes queue state and therefore later routing.

Interf.	Delay (s)	SA-PLB minus ML (ms)	Better in	SA-PLB minus WRR (ms)	Better in
0	0.1	-1	1/3	+94	0/3
0	1.5	-103	3/3	+193	0/3
0	4	-389	3/3	+57	0/3
0.1	0.1	+4	0/3	-556	2/3
0.1	1.5	-83	3/3	-428	2/3
0.1	4	-346	3/3	-460	2/3
0.25	0.1	+8	0/3	-4,195	3/3
0.25	1.5	-75	3/3	-3,901	3/3
0.25	4	+1,136	1/3	-2,099	3/3

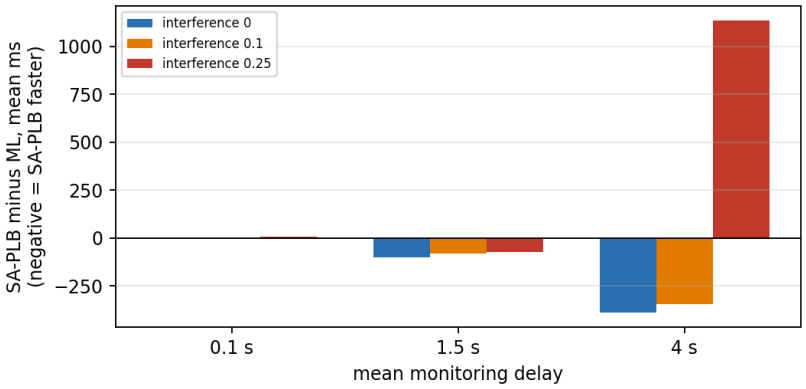


Figure 3. SA-PLB minus ML mean response time (ms).

5.5 Model prediction performance

Trained on 34,814 samples, validated on 6,596 and tested on 5,641. Test MAE was 175.0 ms and RMSE 598.6 ms, against 236.1 ms and 610.0 ms for a median predictor. The MAE gain is clear, but the RMSE gain is marginal, so large errors are not well predicted. The test period is the one with **no interference** (the validation period has interference 0.10), so this test error should not be read as typical for the interference scenarios.

6. Discussion

Staleness is a real problem for ML routing in this setup. With no interference, plain ML's mean response time rises from 397 ms (0.1 s delay) to 748 ms (4 s delay). **The freshness gate helps in most stale conditions:** SA-PLB is faster than ML in all three seeds in five of six stale-data conditions (by 75 to 389 ms, 7 to 52 percent), and is essentially the same as ML when data is fresh (within 8 ms; fallback is almost never triggered at 0.1 s delay). **It is not a universal fix.** With heavy interference (0.25) and a 4 s delay, 80% of decisions fall back to WRR, which handles interference badly, and SA-PLB is about 1,136 ms slower than ML on average (worse in 2 of 3 seeds). The gate is only as good as its fallback. **Versus WRR:** without interference WRR wins in every seed because its exact static weights are hard to beat. With interference, SA-PLB is faster on average, but at 0.10 interference only in 2 of 3 seeds because WRR's results vary widely there; at 0.25 it is faster in all seeds. Most of this advantage comes from ML itself, not from the freshness gate, so the fair comparison for the staleness idea is SA-PLB against ML.

7. Limitations

1. All performance values are simulated; workload shape is the only real input, and its columns are undocumented.
2. Interference rates, delays, service times and replica specifications are assumptions, not calibrated to a real system.
3. Only three test seeds; no significance tests, so close differences are unproven.
4. The model's test period has no interference while validation has some, so its reported error is optimistic for interference regimes.
5. The model is trained on random-routing logs but deployed under its own routing (distribution shift).
6. One threshold was chosen globally; 2.0 s and 4.0 s were nearly tied on validation, and the best threshold likely depends on delay and interference.
7. The only fallback tested is WRR; no 'ML with observation age as a feature' baseline was evaluated.
8. Three replicas and one peak-load level (0.85).

8. Conclusion

SA-PLB adds a freshness check and WRR fallback to ML routing. In this simulation it behaves essentially like ML when monitoring is fresh, improves on ML in five of six stale-data conditions, and fails in one (heavy interference with very stale data). It beats WRR when replicas suffer interference but not when they do not. The

evidence supports the idea that observation freshness matters for predictive routing, and that a freshness gate can reduce staleness harm when the fallback is sound, but not that SA-PLB is generally superior. Broader claims need real traces, more seeds and stronger baselines.

9. Future Work

Validate on real container or cloud traces; add an ML baseline that receives observation age as a feature; test adaptive thresholds and better fallbacks (for example, a least-queue rule); add confidence intervals with more seeds; and include interference in the model's test period.

Appendix: key reported values and reproducibility

Item	Value
Chosen freshness threshold	2.0 s (validation seeds 100, 101)
Test seeds	0, 1, 2
Model test MAE / RMSE	175.0 ms / 598.6 ms
Median baseline MAE / RMSE	236.1 ms / 610.0 ms
Samples (train / val / test)	34,814 / 6,596 / 5,641
Reproduce	python run_all.py --zenodo (about 35 min on one CPU). Running without --zenodo silently uses a synthetic load shape and gives different numbers.